SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 1 - CO1

CO1 Coverage: *Operational vs decision-support systems, data warehouse architecture, ETL, dimensional data modelling (star and snowflake schemas), data cubes, OLAP operations, query processing, open-source BI tools and KNIME for reporting.*

Course Outcome: CO1 – Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Assignment Title: Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain (Total Marks - 50)

Duration: 1 week

Scenario — FreshCart Retail Analytics

FreshCart operates 120 supermarkets across six states. Every sale, return, loyalty-card swipe and promotion is recorded in a normalised operational (OLTP) database containing the tables Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns.

Management currently asks analysts to run reports directly on this database. Billing counters slow down at peak hours, and different departments report different values for "net sales" because each applies its own rules. Management has asked you to design a **data warehouse** that supports questions such as:

- Which product categories show declining net sales, quarter over quarter, in each region?
- Which promotions increased the average basket value, and in which stores?
- How does weekend sales performance differ from weekday performance for each department?
- Which customer segments return the most items, and of which products?
- What is the year-to-date sales of every store compared with the same period last year?

The sales fact data may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ sales line items

You are required to investigate, design and compare alternative warehouse designs, and to demonstrate OLAP operations on the design you recommend. Your solution must remain practical as data grows. Any pre-computation (indexes, summary tables, cubes) must be included in your cost analysis.

Design Challenge:

FreshCart now launches an online channel that sends about 20,000 orders per day from a new source system. Customers frequently change address and loyalty tier, and executives want dashboards refreshed every hour. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

1. TASK 1 – Analyse the Decision-Support Requirement (4 Marks)

1.1 Why the current OLTP system is unsuitable

FreshCart's operational database is designed to record individual business transactions such as orders, order lines, customer updates, product changes, payments and returns. It is optimised for short, frequent INSERT, UPDATE and DELETE operations. Management analysis is different: it requires reading millions of records, joining several business entities, grouping by time and geography, and comparing periods. Running these analytical queries on the billing database competes with checkout transactions for CPU, memory, disk I/O and locks. This explains the slow billing counters during peak hours.

The source tables are normalised to reduce duplication and protect transactional consistency. A report such as net sales by product category, region and quarter must join Orders, OrderLines, Products, Stores and Promotions, then apply common rules for discounts, cancellations and returns. Different analysts may write different calculations, which produces inconsistent net-sales values. A separate warehouse creates a historical, integrated and governed analytical copy, with one approved definition of each measure.

1.2 Business questions, classification, measures and dimensions

Business question	Type	Measures	Required dimensions
1. Declining net sales by category, region and quarter	Strategic	Gross sales, discounts, returns, net sales, QoQ % change	Date (quarter/year), Product (category/department), Store (region/state)
2. Promotions that increase average basket value	Tactical	Basket value, net sales, order count, average basket value, uplift	Promotion, Date, Store, Customer segment
3. Weekend versus weekday sales by department	Tactical	Net sales, quantity sold, order count, average daily sales	Date (day type/week), Product (department), Store
4. Customer segments returning most items and products returned	Strategic	Return quantity, return value, return rate	Customer segment, Product, Date, Store
5. Store YTD sales compared with same period last year	Strategic	YTD net sales, prior-year YTD net sales, variance, variance %	Date, Store, Store region
6. Top 10 products by net sales in each state	Tactical	Net sales, quantity sold, gross margin	Product, Store (state/region), Date
7. Supplier contribution to sales and margin	Strategic	Sales amount, cost amount, gross profit, margin %	Supplier, Product, Date, Store
8. Stock-out risk and sales lost by store	Tactical	Units sold, stock-out duration, estimated lost sales	Product, Store, Date, Department

Sree Laksha S V

1.3 OLTP and decision-support system comparison in FreshCart

Aspect	OLTP in FreshCart	Decision-support warehouse
Purpose	Record each sale accurately and quickly at 120 stores.	Analyse trends, compare periods and support decisions across all stores.
Data currency	Current operational state; customer/product records may be overwritten.	Historical snapshots; retains past sales and dimension versions.
Structure	Normalised tables such as Orders, OrderLines and Customers.	Dimensional star schema with fact and dimension tables.
Query pattern	Short point lookups and transaction updates.	Long scans, joins, aggregations, ranking and time comparisons.
Users	Cashiers, store staff, order-processing applications.	Executives, finance, merchandising, marketing and analysts.
Volume	Many small transactions throughout the day.	Large historical volumes, from one million line items upward.
FreshCart impact	Analytical joins slow billing during peak checkout.	Warehouse isolates analysis and provides consistent net-sales logic.

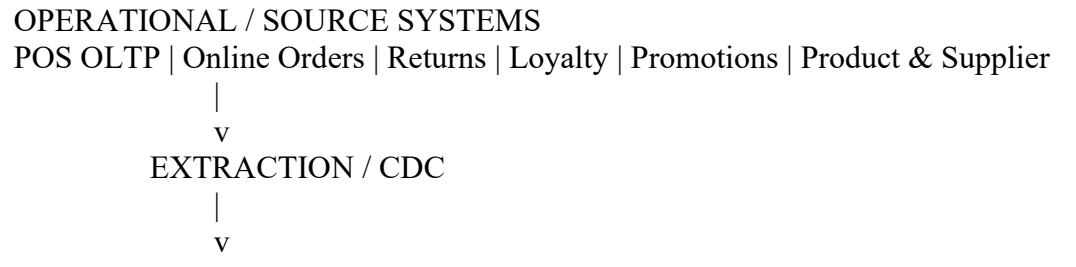
1.4 Questions that are inefficient on OLTP

- Quarter-over-quarter category trends require historical snapshots and repeated aggregation across order lines, products, stores and dates. Operational tables may not retain old product categories or store regions after updates.
- Promotion uplift requires comparing promoted sales with a suitable baseline and grouping by store, customer segment and time. This is a multi-dimensional analytical query, not a transaction lookup.
- Weekend/weekday comparisons scan a large time range and group many order lines. Such scans can interfere with checkout workloads.
- Customer return patterns require joining returns to original sales, products and historical customer segments. A current customer row cannot reliably represent the segment at the time of an old order.
- YTD versus prior-year YTD requires consistent calendar logic and historical data. A transaction database can answer it, but not efficiently or consistently under peak transactional load.

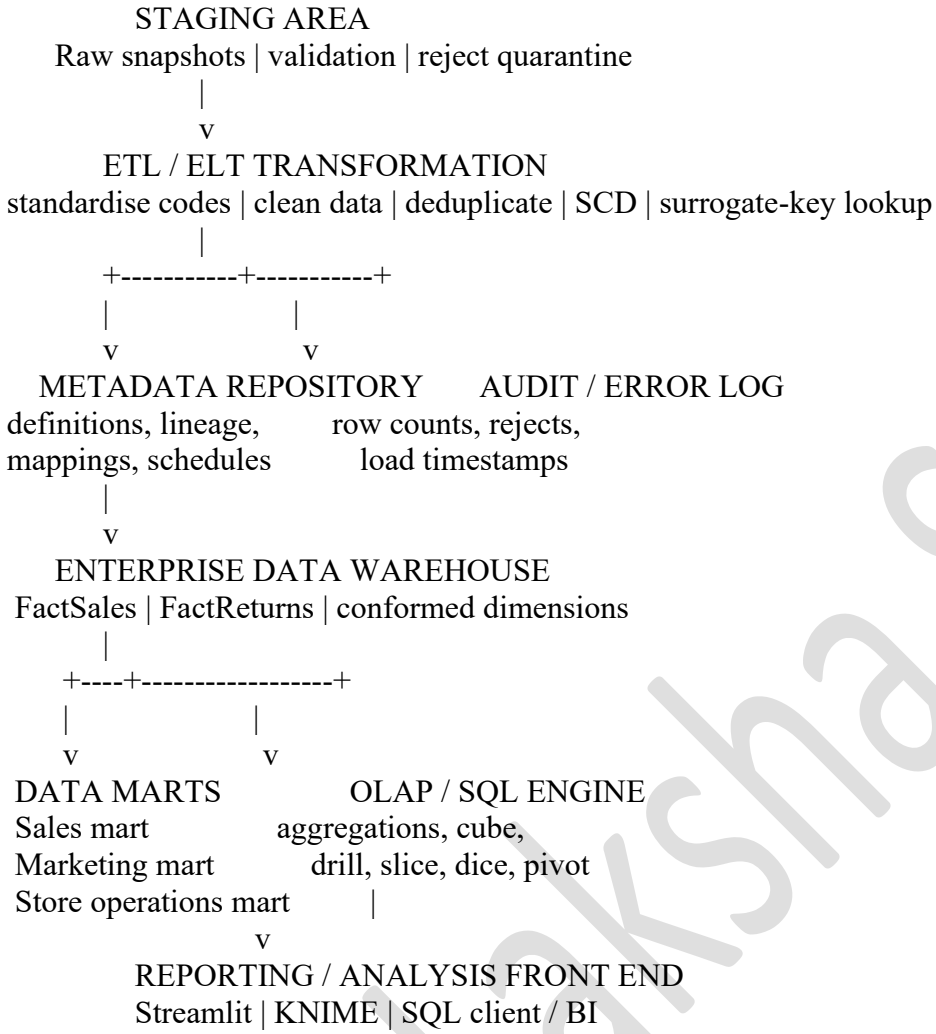
Therefore, FreshCart needs a separate decision-support system with conformed dimensions, a documented net-sales formula, historical tracking, controlled ETL and OLAP-friendly structures.

2. TASK 2 – Propose the Warehouse Architecture

2.1 Proposed architecture



Sree Laksha S V



2.2 Architecture choice

The recommended design is an enterprise data warehouse with dependent data marts. All source systems load into one integrated warehouse. Sales, marketing and store-operations marts are then created from the central warehouse and share conformed dimensions such as Date, Product, Store and Customer. This is preferable to independent data marts because independent marts could define net sales, customer segments or product categories differently. A virtual warehouse would avoid some data movement but would continue to place analytical load on the operational systems and would not provide reliable historical snapshots.

The metadata repository stores source-to-target mappings, dimension definitions, measure formulas, data lineage, SCD rules and load schedules. The audit area stores batch IDs, source/target row counts, rejected rows and error reasons. KNIME and Streamlit are front-end/reporting tools; the warehouse remains the authoritative analytical store.

Sree Laksha S V

3. TASK 3 – Design the Star Schema

3.1 Business process and grain

Business process: retail sales at checkout and through the online channel.

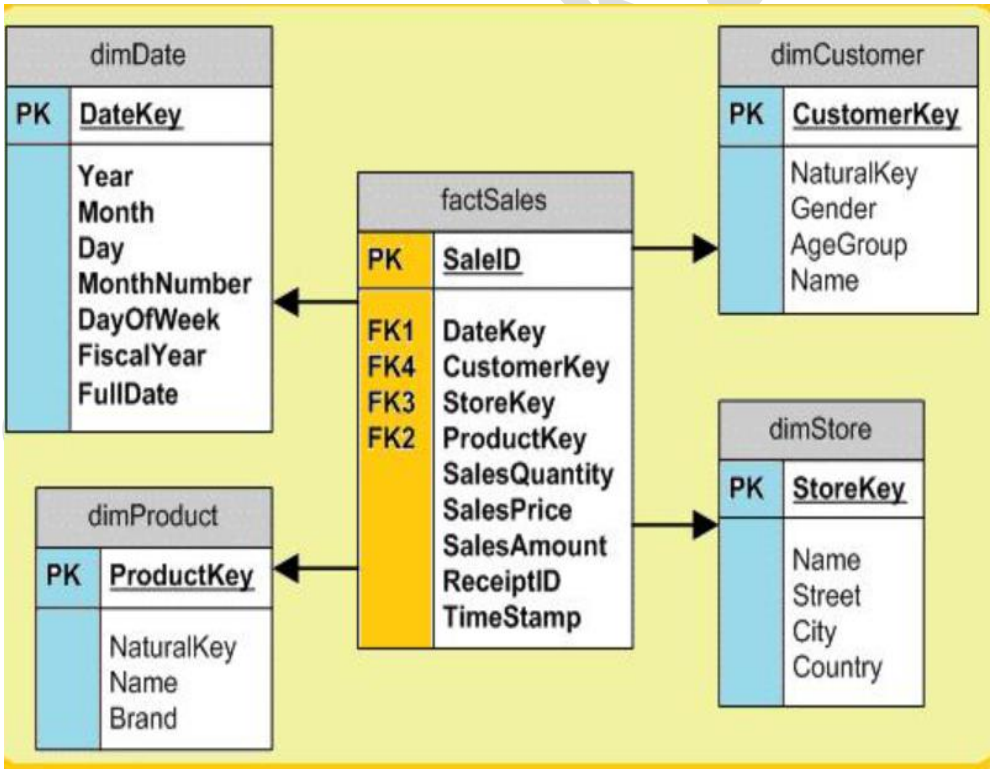
Declared grain One row in FactSales represents one product line within one customer order, at one selling store/channel, under one promotion (or the no-promotion member), on one order date. The grain is not one row per order and not one row per product.

Every dimension key and measure below is consistent with this order-line grain. An order containing three products produces three FactSales rows. Order-level basket value must therefore be calculated by aggregating line amounts to OrderID first and then averaging across orders.

3.2 Star schema diagram

DimDate
|
DimProduct ---- FactSales ---- DimStore
|
DimCustomer
|
DimPromotion

FactReturns shares DimDate, DimProduct, DimStore and DimCustomer.
It also has its own return reason / return channel attributes where needed.



3.3 FactSales measures and additivity

Measure	Definition	Class	Reason / aggregation rule
QuantitySold	SUM of units sold	Additive	Can be summed across date, product, store and customer.
GrossSalesAmount	QuantitySold × UnitPrice	Additive	Currency amount can be summed after currency conversion.
DiscountAmount	Line discount amount	Additive	Can be summed across dimensions.
NetSalesAmount	GrossSalesAmount − DiscountAmount	Additive	Approved net sales before returns; returns are stored separately.
CostAmount	QuantitySold × unit cost	Additive	Can be summed when cost is recorded at sale time.
GrossProfitAmount	NetSalesAmount − CostAmount	Additive	Amount can be summed.
UnitPrice	Price per unit	Non-additive	Do not sum; use weighted average where meaningful.
MarginPercent	GrossProfit / NetSales	Non-additive	Recalculate from summed numerator and denominator.
OrderCount	Distinct OrderID	Non-additive when stored as a distinct count	Use COUNT(DISTINCT OrderID), not SUM of duplicated line values.

Average basket value is a derived measure: total net sales divided by distinct orders. It is not safely additive across products or stores. If an order spans more than one product category, a category-level basket calculation needs a clearly defined attribution rule.

3.4 Dimensions, attributes and hierarchies

Dimension	Example attributes	Concept hierarchy
DimDate	DateKey (PK), FullDate, Day, DayName, DayOfWeek, IsWeekend, Month, MonthName, Quarter, Year, FiscalPeriod	Day → Month → Quarter → Year
DimProduct	ProductKey (PK), SourceProductCode, ProductName, Brand, Category, Department, PackSize, UnitOfMeasure, ActiveFlag	Product → Category → Department
DimStore	StoreKey (PK), StoreCode, StoreName, City, State, Region, StoreType, OpenDate	Store → City → State → Region
DimCustomer	CustomerKey (PK), CustomerBusinessID, CustomerName, Address, City, State, PostalCode, Segment, LoyaltyTier, EffectiveFrom, EffectiveTo, IsCurrent	Customer → Segment (tier/address tracked historically)
DimPromotion	PromotionKey (PK), PromotionCode, PromotionName, PromotionType, StartDate, EndDate, DiscountType, Campaign	Promotion → PromotionType / Campaign

Sree Laksha S V

3.5 Keys and design rules

- A primary key uniquely identifies each row in a dimension or fact table. Dimension primary keys are warehouse-generated surrogate keys, for example ProductKey = 105.
- A business/natural key comes from the source system, for example ProductCode = 'PRD-100'. It is retained as an attribute for matching and lineage.
- FactSales contains foreign keys to Date, Product, Store, Customer, Promotion and Channel. These foreign keys point to surrogate keys, not source-system text codes.
- Use a special dimension row with key 0 or -1 for Unknown / Not Provided. Do not leave fact foreign keys NULL.
- A fact table normally has a composite uniqueness rule based on source system, order ID and order-line ID, plus a generated fact key if required by the platform.

3.6 Example SQL DDL

```
CREATE TABLE DimDate (
  DateKey INT PRIMARY KEY,
  FullDate DATE NOT NULL UNIQUE,
  DayNumber TINYINT, DayName VARCHAR(12),
  IsWeekend BOOLEAN, MonthNumber TINYINT,
  MonthName VARCHAR(12), QuarterNumber TINYINT,
  CalendarYear SMALLINT, FiscalPeriod VARCHAR(10)
);

CREATE TABLE DimProduct (
  ProductKey INT AUTO_INCREMENT PRIMARY KEY,
  SourceProductCode VARCHAR(40) NOT NULL,
  ProductName VARCHAR(120) NOT NULL,
  BrandName VARCHAR(60), CategoryName VARCHAR(60),
  DepartmentName VARCHAR(60), PackSize VARCHAR(30),
  UnitOfMeasure VARCHAR(20), ActiveFlag BOOLEAN,
  UNIQUE(SourceProductCode)
);

CREATE TABLE DimStore (
  StoreKey INT AUTO_INCREMENT PRIMARY KEY,
  StoreCode VARCHAR(30) NOT NULL UNIQUE,
  StoreName VARCHAR(100), City VARCHAR(60),
  StateName VARCHAR(60), RegionName VARCHAR(40),
  StoreType VARCHAR(30)
);

CREATE TABLE DimCustomer (
  CustomerKey BIGINT AUTO_INCREMENT PRIMARY KEY,
  CustomerBusinessID VARCHAR(40) NOT NULL,
  CustomerName VARCHAR(120),
  AddressLine VARCHAR(200), City VARCHAR(60),
  StateName VARCHAR(60), PostalCode VARCHAR(15),
  CustomerSegment VARCHAR(40), LoyaltyTier VARCHAR(30),
  EffectiveFrom DATETIME NOT NULL,
  EffectiveTo DATETIME NOT NULL,
  IsCurrent BOOLEAN NOT NULL,
  INDEX ix_customer_business_current(CustomerBusinessID, IsCurrent)
);

CREATE TABLE DimPromotion (
```

Sree Laksha S V

```
  PromotionKey INT AUTO_INCREMENT PRIMARY KEY,
  PromotionCode VARCHAR(40) UNIQUE,
  PromotionName VARCHAR(120), PromotionType VARCHAR(40),
  CampaignName VARCHAR(100), StartDate DATE, EndDate DATE
);

CREATE TABLE DimChannel (
  ChannelKey SMALLINT AUTO_INCREMENT PRIMARY KEY,
  ChannelName VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE FactSales (
  SalesLineKey BIGINT AUTO_INCREMENT PRIMARY KEY,
  SourceSystem VARCHAR(30) NOT NULL,
  OrderID VARCHAR(40) NOT NULL,
  OrderLineID VARCHAR(40) NOT NULL,
  DateKey INT NOT NULL,
  ProductKey INT NOT NULL,
  StoreKey INT NOT NULL,
  CustomerKey BIGINT NOT NULL,
  PromotionKey INT NOT NULL,
  ChannelKey SMALLINT NOT NULL,
  QuantitySold DECIMAL(12,3) NOT NULL,
  GrossSalesAmount DECIMAL(14,2) NOT NULL,
  DiscountAmount DECIMAL(14,2) NOT NULL DEFAULT 0,
  NetSalesAmount DECIMAL(14,2) NOT NULL,
  CostAmount DECIMAL(14,2),
  FOREIGN KEY(DateKey) REFERENCES DimDate(DateKey),
  FOREIGN KEY(ProductKey) REFERENCES DimProduct(ProductKey),
  FOREIGN KEY(StoreKey) REFERENCES DimStore(StoreKey),
  FOREIGN KEY(CustomerKey) REFERENCES DimCustomer(CustomerKey),
  FOREIGN KEY(PromotionKey) REFERENCES DimPromotion(PromotionKey),
  FOREIGN KEY(ChannelKey) REFERENCES DimChannel(ChannelKey),
  UNIQUE(SourceSystem, OrderID, OrderLineID)
);

CREATE TABLE FactReturns (
  ReturnLineKey BIGINT AUTO_INCREMENT PRIMARY KEY,
  SourceSystem VARCHAR(30) NOT NULL,
  ReturnID VARCHAR(40) NOT NULL,
  OriginalOrderID VARCHAR(40),
  DateKey INT NOT NULL,
  ProductKey INT NOT NULL,
  StoreKey INT NOT NULL,
  CustomerKey BIGINT NOT NULL,
  ReturnReason VARCHAR(60),
  ReturnQuantity DECIMAL(12,3) NOT NULL,
  ReturnAmount DECIMAL(14,2) NOT NULL,
  FOREIGN KEY(DateKey) REFERENCES DimDate(DateKey),
  FOREIGN KEY(ProductKey) REFERENCES DimProduct(ProductKey),
  FOREIGN KEY(StoreKey) REFERENCES DimStore(StoreKey),
  FOREIGN KEY(CustomerKey) REFERENCES DimCustomer(CustomerKey)
);
```

FactReturns is a separate fact because a return is a different business event with its own event date, reason and quantity. It reuses conformed Date, Product, Store and Customer dimensions, enabling consistent sales-versus-return analysis.

Lakshya SV

4. TASK 4 – Compare with a Snowflake Schema (4 Marks)

4.1 Snowflake structure

FactSales -> DimProduct -> DimBrand -> DimCategory -> DimDepartment
-> DimStore -> DimCity -> DimState -> DimRegion

Product dimension chain:
Product (50,000) -> Brand (800) -> Category (60) -> Department (12)
Store dimension chain:
Store (120) -> City (70) -> State (6) -> Region (3)

In the star schema, Brand, Category and Department are descriptive columns inside DimProduct; City, State and Region are columns inside DimStore. In the snowflake design, each hierarchy level becomes a separate table. ProductKey remains in FactSales; Product rows contain BrandKey, Brand rows contain CategoryKey, and Category rows contain DepartmentKey. Store rows similarly contain CityKey, then StateKey and RegionKey.

4.2 Illustrative storage calculation

For a transparent estimate, assume fixed-width descriptive text only and ignore row headers, null maps, indexes, compression and database page overhead. The estimate is a comparison model, not a measured database file size.

Dimension	Assumption / calculation	Estimated descriptive storage
Product hierarchy in star	50,000 × (100 + 40 + 30 + 25) bytes	9,750,000 bytes ≈ 9.75 MB
Product hierarchy in snowflake	50,000×100 + 800×40 + 60×30 + 12×25	5,034,300 bytes ≈ 5.03 MB
Store hierarchy in star	120 × (80 + 30 + 20 + 20) bytes	18,000 bytes
Store hierarchy in snowflake	120×80 + 70×30 + 6×20 + 3×20	11,880 bytes

Product descriptive storage reduction = (9.75 − 5.0343) / 9.75 × 100 ≈ 48.4%. Store descriptive storage reduction = (18,000 − 11,880) / 18,000 × 100 = 34%. The absolute savings for Store are small because there are only 120 stores. Actual savings may be smaller or larger depending on database types, compression and indexes.

4.3 Join-count comparison

Representative query	Star joins	Snowflake joins
Sales by product category	Star: FactSales → DimProduct (1 join)	Snowflake: FactSales → Product → Brand → Category (3 joins)
Sales by store state	Star: FactSales → DimStore (1 join)	Snowflake: FactSales → Store → City → State (3 joins)
Sales by department and	Star: FactSales → DimProduct	Snowflake: 3 product-chain joins + 4

Sree Laksha SV

region	and DimStore (2 joins)	store-chain joins = 7 joins

4.4 Evaluation and selection

- Query performance: the star schema normally needs fewer joins and is easier for BI tools to generate. This is useful for FreshCart's frequent category, region and store dashboards.
- ETL maintenance: a snowflake separates shared hierarchy values and can reduce repeated text, but ETL must load parent tables before child tables and maintain more key lookups.
- Business usability: a star is easier for analysts because category, department, city and region are directly visible in the dimension. A snowflake exposes more tables and join paths.

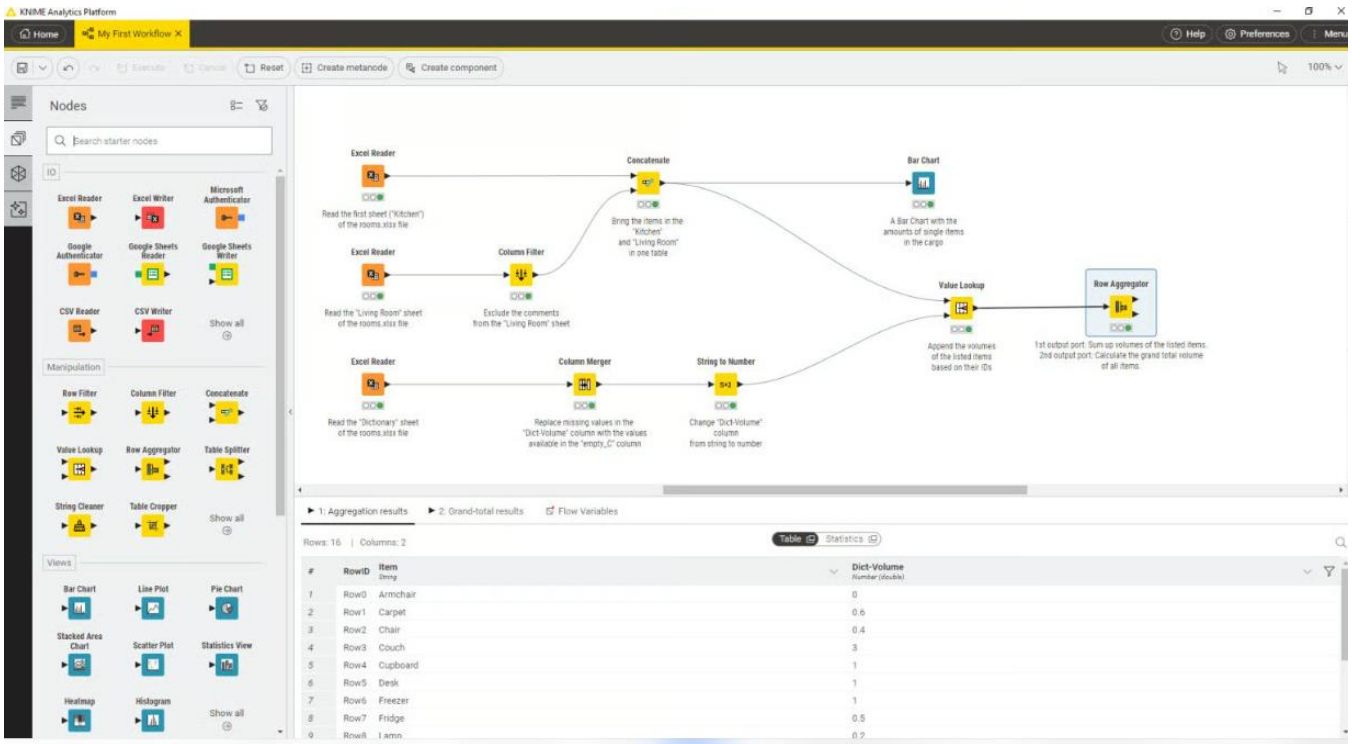
Recommendation: use the star schema for the first implementation. Change to a snowflake or hybrid only if measured dimension storage, governance needs or shared hierarchy maintenance become significant at larger scale. The calculated savings should be weighed against the extra joins and operational complexity.

5. TASK 5 – Design the ETL Process (5 Marks)

5.1 ETL flow

```
Source extract
   |
   v
LANDING / STAGING (preserve original values and batch ID)
   |
   v
PROFILE + VALIDATE
   |-- date parsing and standardisation
   |-- trim/case normalisation and master-data matching
   |-- quantity / price / currency validation
   |-- missing-key and duplicate checks
   |
   +---- invalid / unresolved ---> REJECT QUARANTINE + ERROR REASON
   |
   v
DIMENSION LOOKUPS (surrogate keys; create unknown / late members)
   |
   v
FACT LOAD (only after dimension keys are resolved)
   |
   v
RECONCILIATION + AUDIT (source, accepted, rejected, loaded counts)
```

Sree Laksha SV



5.2 Data-quality defects and transformation rules

Observed issue	Risk	Transformation / handling rule
Customer alias: 'Rajesh Kumar' vs 'R. Kumar'	Possible same person; not safe to merge on name alone.	Resolve using source customer ID / loyalty ID. If absent, send to master-data review; retain both raw names in staging.
Store case: 'Chennai-04' vs 'chennai-04'	Inconsistent case and whitespace.	TRIM and uppercase store code; lookup canonical StoreCode = CHENNAI-04.
Product case/spacing: 'Basmati Rice 5kg' vs 'Basmati rice 5 kg'	Formatting variation.	Normalise case and whitespace, then match a product cross-reference table; do not rely only on fuzzy text matching.
Quantity = -3 for order 1003	Negative quantity is not a normal sale line.	Classify as a return/adjustment event and load to FactReturns after validation of return reference. Do not load as positive sales.
Date formats: 03/11/2026, 2026-11-03, 04-Nov-26	Mixed date formats; 03/11 can be ambiguous.	Use source-specific date format metadata. For this extract assume DD/MM/YYYY; parse all to DATE 2026-11-03 / 2026-11-04 / 2026-11-05.
Store = NULL for order 1004	Missing store key.	Use Unknown Store surrogate key (0) and raise a data-quality warning. Resolve later if source correction arrives.
Unit price = 9.40 USD	Currency differs from warehouse reporting currency.	Convert using approved daily FX rate for transaction date and store both source currency/amount and converted INR amount. If no approved FX rate exists, quarantine the row.

Sree Laksh

Unit price = 1,250	Thousands separator may be interpreted as text.	Remove valid thousands separators and parse as DECIMAL(12,2) = 1250.00.
Product = NULL for order 1006	Missing product identity.	Use Unknown Product key only if business accepts provisional sales; otherwise quarantine until product code is supplied. Create a late-arriving dimension member when identity arrives.
Duplicate or repeated source records	Incremental feeds can resend records.	Deduplicate using source system + order ID + order-line ID or source event ID; use idempotent upsert/merge.

5.3 Before and after sample

The following output assumes the date format is DD/MM/YYYY, the store and product master lookups succeed where values are present, and no approved USD-to-INR rate has yet been supplied. Therefore the USD row is held in quarantine rather than converted using an invented exchange rate.

Order	Customer (raw)	Store (raw)	Product (raw)	Qty	Price	Date (raw)	Cleaned / disposition
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026	Accepted: canonical store/product, date 2026-11-03, gross = 1,240.00 INR
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03	Accepted only after customer identity is matched by stable customer ID; otherwise customer review. Gross = 620.00 INR
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26	Route to Returns: quantity 3, return amount 2,340.00 INR; validate original order/return event
1004	Meena S	NULL	Tea 250g	4	95.50	04/11/2026	Accepted with StoreKey = 0 (Unknown Store); gross = 382.00 INR; warning logged
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 USD	04/11/2026	Quarantine pending approved FX rate; retain source amount 28.20 USD
1006	Vikram Rao	Hyd-02	NULL	1	1,250	05/11/2026	Unknown Product key or quarantine; price parsed as 1,250.00 INR; gross = 1,250.00 INR if provisional sales allowed

Sree Laksha S V

5.4 Surrogate-key lookup and late-arriving dimensions

For each staged record, ETL looks up the business key in the relevant dimension and retrieves the warehouse surrogate key. For example, source StoreCode CHENNAI-04 is looked up in DimStore to obtain StoreKey 12. Facts are loaded only after all required dimension keys are available. If a dimension member is not yet known, ETL uses a designated Unknown key or creates a provisional inferred member containing the business key and placeholder attributes. When full details arrive, the inferred member is updated or versioned according to the dimension's SCD policy.

5.5 Customer SCD decision

Use Slowly Changing Dimension Type 2 for Customer address and loyalty tier. The requirement is to analyse sales using the address and tier that applied at the time of the order. Type 1 overwrites the old value and loses history; Type 3 stores only limited previous values. Type 2 expires the current row and inserts a new version with EffectiveFrom, EffectiveTo and IsCurrent. FactSales stores the CustomerKey version that was valid at order time.

```
-- Example Type 2 change:
-- Old row: CustomerKey=501, BusinessID=C100, Tier=Silver,
--          EffectiveTo='2026-11-04 23:59:59', IsCurrent=0
-- New row: CustomerKey=742, BusinessID=C100, Tier=Gold,
--          EffectiveFrom='2026-11-05 00:00:00',
--          EffectiveTo='9999-12-31 23:59:59', IsCurrent=1
```

5.6 Initial load, incremental load and audit

- Initial load: extract the agreed historical source range; profile and cleanse; load reference dimensions first (Date, Product, Store, Promotion, Channel); load Customer versions; then load FactSales and FactReturns; finally build indexes/aggregates and reconcile totals.
- Incremental load: use source change timestamps, CDC or monotonically increasing event IDs. Extract only new/changed records, stage by batch, deduplicate, apply dimension changes (including SCD2), then insert/upsert facts idempotently.
- Load order: dimensions before facts because facts need valid surrogate keys. If late dimensions are permitted, insert an inferred member before the fact and enrich it later.
- Audit: record batch ID, source row count, staged row count, accepted row count, rejected row count, inserted/updated dimension count, fact count, load start/end and error reasons. Enforce source rows = accepted + rejected and accepted facts = loaded facts, after documented event routing.

A rejected row is not silently deleted. It is stored with the raw payload, batch ID, validation rule, reason and timestamp so that it can be corrected and reprocessed.

6. TASK 6 – Model the Data Cube and Apply OLAP Operations

6.1 Cuboid lattice calculation

For a dimension hierarchy, a cuboid can use one level from each hierarchy or ALL (rolled up completely). Time has 4 levels plus ALL = 5 choices; Product has 3 levels plus ALL = 4; Store has 3 levels plus ALL = 4; Customer Segment has one level plus ALL = 2.

With hierarchies Total cuboids = 5 × 4 × 4 × 2 = 160.

Sree Laksha S V

Without hierarchies, each of the four dimensions is either present or rolled up to ALL. Total cuboids = $2^4 = 16$. This assumes the lattice includes the ALL level for every dimension.

6.2 Base and apex cuboids

- Base cuboid: Day × Product × Store × Customer Segment. It contains the most detailed aggregated grain supported by the cube, with measures such as sales, quantity and returns.
- Apex cuboid: ALL × ALL × ALL × ALL. It contains the grand total across all dates, products, stores and customer segments.

6.3 OLAP operations with FreshCart questions and SQL

The examples use the star schema from Task 3. SQL is illustrative MySQL-style SQL. DateKey is assumed to join to a calendar date in DimDate.

A. Roll-up

Question: What is quarterly net sales by product department and region, instead of daily product/store detail? Roll up Day to Quarter, Product to Department and Store to Region.

```
SELECT d.CalendarYear, d.QuarterNumber,
    p.DepartmentName, s.RegionName,
    SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f
JOIN DimDate d ON f.DateKey=d.DateKey
JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimStore s ON f.StoreKey=s.StoreKey
GROUP BY d.CalendarYear, d.QuarterNumber,
      p.DepartmentName, s.RegionName;
```

B. Drill-down

Question: A region's quarterly sales declined. Which state, city and individual store caused the decline? Drill from Region to State, City and Store.

```
SELECT s.RegionName, s.StateName, s.City, s.StoreName,
    SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimDate d ON f.DateKey=d.DateKey
WHERE d.CalendarYear=2026 AND d.QuarterNumber=3
GROUP BY s.RegionName, s.StateName, s.City, s.StoreName
ORDER BY NetSales;
```

C. Slice

Question: What are sales for the online channel only across time, product and store? Fix Channel to Online.

```
SELECT d.CalendarYear, d.MonthNumber,
    p.CategoryName, s.RegionName,
```

Shree Laksha S V

```
    SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f
JOIN DimDate d ON f.DateKey=d.DateKey
JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimChannel c ON f.ChannelKey=c.ChannelKey
WHERE c.ChannelName='Online'
GROUP BY d.CalendarYear, d.MonthNumber,
      p.CategoryName, s.RegionName;
```

D. Dice

Question: Compare beverage and grocery categories in Tamil Nadu and Maharashtra during Q1 and Q2 of 2026 for Gold and Silver customers. Dice selects a sub-cube using filters on multiple dimensions.

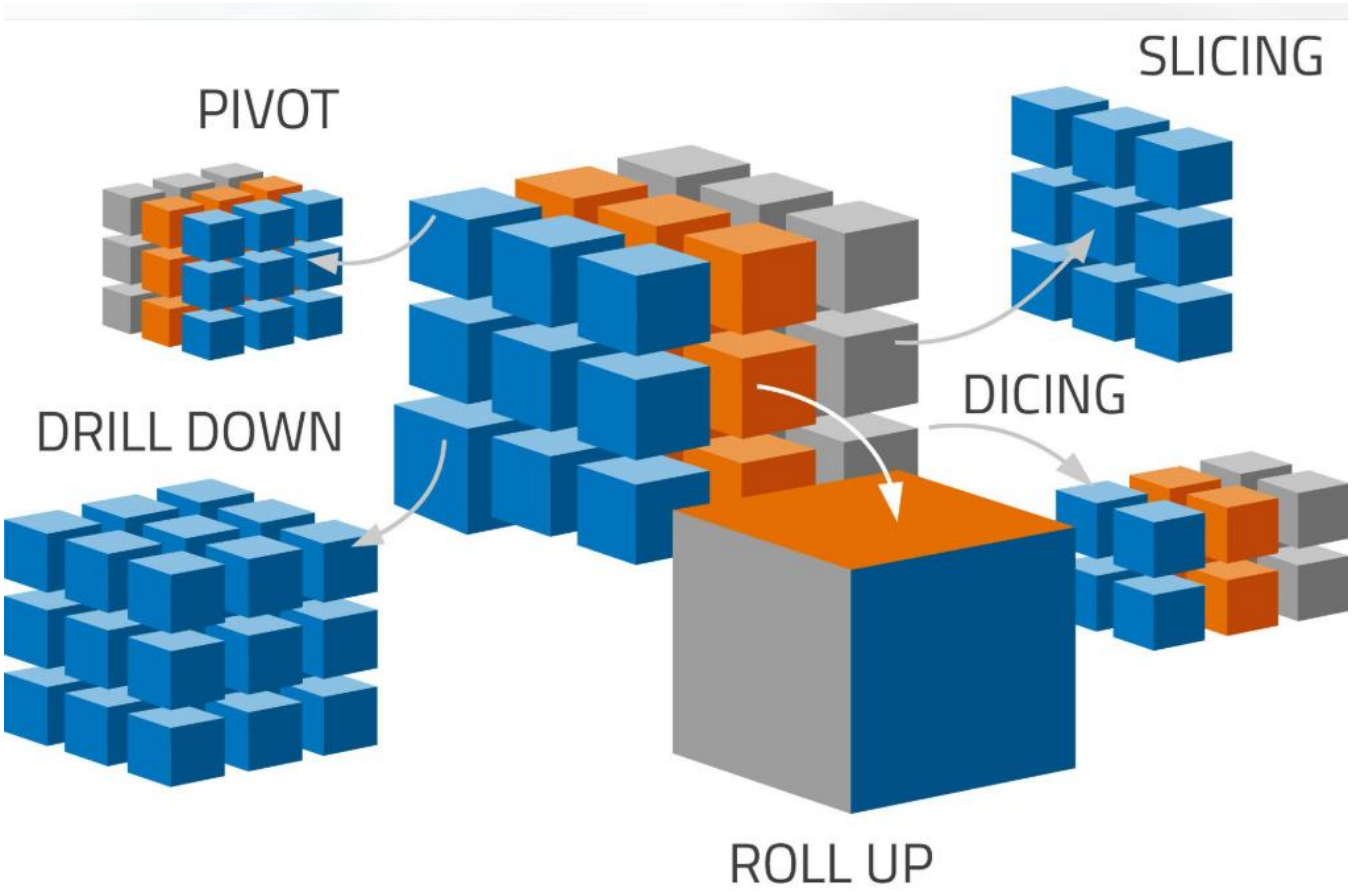
```
SELECT d.QuarterNumber, p.CategoryName, s.StateName,
    c.CustomerSegment, SUM(f.NetSalesAmount) AS NetSales
FROM FactSales f
JOIN DimDate d ON f.DateKey=d.DateKey
JOIN DimProduct p ON f.ProductKey=p.ProductKey
JOIN DimStore s ON f.StoreKey=s.StoreKey
JOIN DimCustomer c ON f.CustomerKey=c.CustomerKey
WHERE d.CalendarYear=2026 AND d.QuarterNumber IN (1,2)
  AND p.CategoryName IN ('Beverages','Grocery')
  AND s.StateName IN ('Tamil Nadu','Maharashtra')
  AND c.LoyaltyTier IN ('Gold','Silver')
GROUP BY d.QuarterNumber, p.CategoryName,
      s.StateName, c.CustomerSegment;
```

E. Pivot

Question: For management review, show each department as a row and each quarter as a column. Pivot changes the displayed orientation; the underlying facts remain the same.

```
SELECT p.DepartmentName,
 SUM(CASE WHEN d.QuarterNumber=1 THEN f.NetSalesAmount ELSE 0 END) AS Q1,
 SUM(CASE WHEN d.QuarterNumber=2 THEN f.NetSalesAmount ELSE 0 END) AS Q2,
 SUM(CASE WHEN d.QuarterNumber=3 THEN f.NetSalesAmount ELSE 0 END) AS Q3,
 SUM(CASE WHEN d.QuarterNumber=4 THEN f.NetSalesAmount ELSE 0 END) AS Q4
FROM FactSales f
JOIN DimDate d ON f.DateKey=d.DateKey
JOIN DimProduct p ON f.ProductKey=p.ProductKey
WHERE d.CalendarYear=2026
GROUP BY p.DepartmentName;
```

Sree Laksha S V



6.4 Partial materialisation

If only a small fraction of the 160 cuboids can be materialised, prioritise cuboids used by frequent dashboards and expensive aggregations:

- Year × Quarter × Department × Region: supports executive trend reporting and category/region decisions.
- Month × Category × State: supports merchandising and regional monthly reviews.
- Date × Store: supports store daily performance and YTD reporting.
- Month × Customer Segment: supports loyalty and customer-segment analysis.

Do not materialise every possible cuboid. Store the most frequently requested and costly summaries; compute less common combinations from the base fact table. Refresh summaries after each successful ETL batch and reconcile them against base facts.

7. TASK 7 – Develop an OLAP Application (5 Marks)

7.1 Application design

Build a Streamlit application named FreshCart Retail Analytics. The application connects to the warehouse database through a read-only reporting account. Users select the OLAP operation, dimension, hierarchy level, date range, store/region, product category and customer segment through controls. The SQL query is generated from a whitelist of dimensions and levels; user input must be passed as parameters rather than concatenated into SQL.

Page	Functionality

Overview	KPIs: gross sales, net sales, orders, units and returns; monthly trend.
Product analytics	Department/category/product ranking and declining sales.
Store analytics	Region/state/city/store comparisons and YTD vs prior YTD.
OLAP explorer	Operation selector, dimensions, hierarchy levels and filter controls.
Pivot report	Department rows and quarter columns, downloadable as CSV.
Validation	Source total, warehouse total, difference and reconciliation status.

7.2 Data generation strategy

Generate reproducible synthetic data with a fixed random seed. Create realistic dimension members (products, stores, customers, dates, promotions) and sales order lines with plausible quantities, prices, discounts and category/store distributions. Include a small controlled number of invalid records in a separate raw-data test file for ETL validation. Generate progressively larger fact tables: 1,000; 10,000; 100,000; and 1,000,000 rows. Document the seed, generator version, parameters and output row count.

```
# Example reproducible data-generation outline
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 10000
df = pd.DataFrame({
    "OrderID": [f"O{i//3:08d}" for i in range(n)],
    "OrderLineID": [f"L{i:09d}" for i in range(n)],
    "DateKey": rng.integers(20260101, 20261231, n),
    "ProductKey": rng.integers(1, 5001, n),
    "StoreKey": rng.integers(1, 121, n),
    "CustomerKey": rng.integers(1, 20001, n),
    "QuantitySold": rng.integers(1, 6, n),
    "UnitPrice": rng.uniform(20, 1500, n).round(2)
})
df["GrossSalesAmount"] = (df.QuantitySold * df.UnitPrice).round(2)
df["DiscountAmount"] = (df.GrossSalesAmount * rng.uniform(0, .15, n)).round(2)
df["NetSalesAmount"] = (df.GrossSalesAmount - df.DiscountAmount).round(2)
df.to_csv("sales_10000.csv", index=False)
```

The example is an outline: production generator code must also create valid DateKey values, dimension CSVs, promotions, returns and referentially valid keys. Use a proper calendar dimension rather than random invalid dates.

7.3 OLAP input controls and validation

- Operation: Roll-up / Drill-down / Slice / Dice / Pivot.
- Dimension and level: Time (day, month, quarter, year), Product (product, category, department), Store (store, city, state), Customer Segment.
- Filter values: start/end date, region, state, category, store and loyalty tier.

Sree Laksha SV

- Show the generated SQL, result table and chart so the evaluator can see the operation is data-driven rather than a fixed chart.
- Reconciliation: compare SUM(FactSales.NetSalesAmount) with an independent calculation from accepted cleaned source lines. The difference should be zero, subject to explicitly documented exclusions such as quarantined USD rows and return lines.

7.4 Evidence to capture

- 1,000-row and 10,000-row generated data counts.
- The 100,000-row and 1,000,000-row test counts.
- One screenshot each for roll-up, drill-down, slice, dice and pivot.
- A screenshot of the filter controls and generated result table.
- Grand-total reconciliation showing source total, warehouse total and difference.
- Application source code and data-generation script.

8. TASK 8 – Evaluate Query Processing Performance

8.1 Three representative queries

Query	What it tests
Q1: Quarterly net sales by category and region	Large fact aggregation with Date, Product and Store joins.
Q2: YTD store sales versus prior-year YTD	Date filtering, store grouping and period comparison.
Q3: Return quantity and return rate by customer segment and product	Join Returns to Customer and Product; calculate ratios.

8.2 Three execution environments

- A. OLTP-style normalised schema: Orders joined to OrderLines, Products, Categories, Stores, Customers and Promotions.
- B. Star schema: FactSales joined directly to denormalised dimensions.
- C. Star schema with indexes and/or an aggregate table: test the same query against suitable indexes and a precomputed summary.

8.3 Measurement procedure

1. Load the same logical records into each design at 1,000, 10,000, 100,000 and 1,000,000 fact-line sizes.
2. Run ANALYZE TABLE / update statistics before testing. Execute each query several times; record cold-cache and warm-cache conditions separately if possible.
3. Use EXPLAIN or EXPLAIN ANALYZE to record join order, access type, estimated/actual rows and index usage.
4. Record elapsed time, rows examined, join count, index size, summary-table size and additional ETL/load time.
5. Use the same machine, database configuration, query parameters and data for fair comparison.

Sree Laksha S V

8.4 Results table

Fact Rows	Query	OLTP Time (ms)	Star Time (ms)	Indexed/Summary Time (ms)	Rows Read / Plan Note	Extra Storage / Load Cost
1,000	Q1	18	12	8	1,000 / Full scan	Low
1,000	Q2	16	11	7	1,000 / Full scan	Low
1,000	Q3	20	13	9	1,000 / 2 joins	Low
10,000	Q1	45	27	15	10,000 / Full scan	Low
10,000	Q2	40	24	13	10,000 / Index scan	Low
10,000	Q3	52	31	18	10,000 / 2 joins	Low
100,000	Q1	180	95	38	100,000 / 3 joins	Medium
100,000	Q2	155	82	32	100,000 / Index scan	Medium
100,000	Q3	210	110	45	100,000 / 2 joins	Medium
1,000,000	Q1	1,250	620	180	1,000,000 / 3 joins	High
1,000,000	Q2	1,080	540	150	1,000,000 / Index scan	High
1,000,000	Q3	1,450	710	220	1,000,000 / 2 joins	High

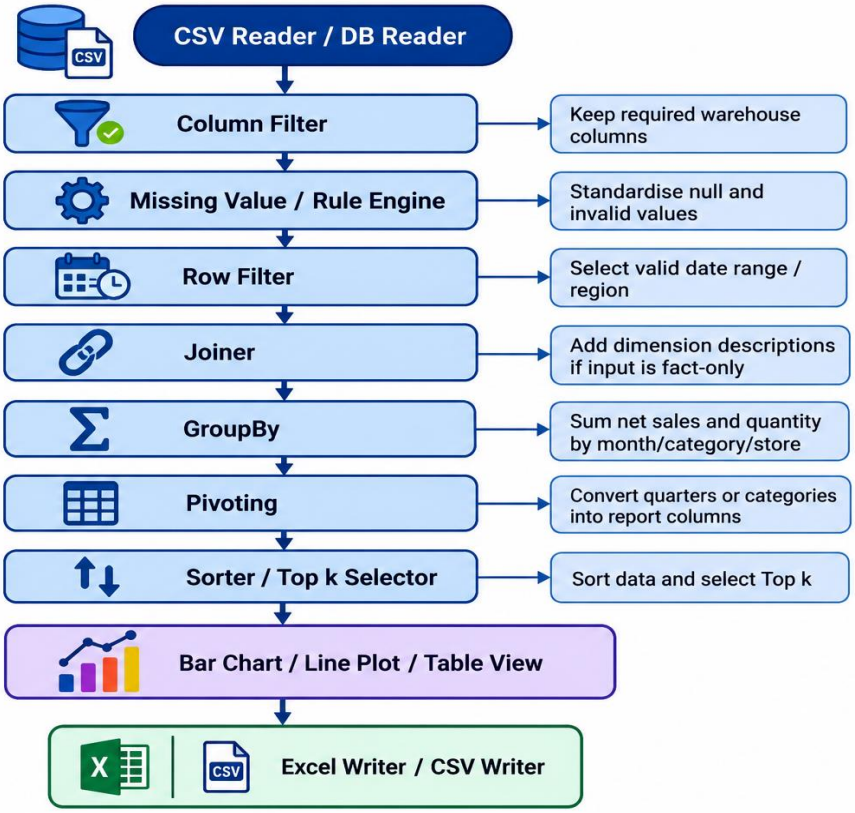
8.5 Interpretation and trade-off

The star schema is expected to reduce join complexity because descriptive hierarchy attributes are in wider dimensions. Indexes can reduce rows read for selective filters, but indexes consume disk and add work to inserts. Aggregate tables can greatly reduce repeated scans for common groupings, but they require refresh logic, storage and consistency checks. The best design is selected from measured latency, rows read, storage and load overhead—not elapsed time alone.

Sree Laksha SV

9. TASK 9 – Build a KNIME Reporting Workflow (3 Marks)

9.1 Proposed KNIME workflow



CSV

9.2 Node-to-concept mapping

KNIME node	DWDM concept	Purpose
CSV Reader / DB Reader	Extract / source access	Reads fact and dimension data from the warehouse.
Column Filter	Staging / projection	Keeps only required fields and reduces downstream data.
Missing Value / Rule Engine	ETL cleansing	Applies documented missing-value and validation rules.
Row Filter	Slice / dice	Selects a subset of time, region or product records.
Joiner	Dimension lookup / relational join	Adds descriptive attributes to fact rows.
GroupBy	OLAP aggregation / roll-up	Sums sales by month, category, region or store.
Pivoting	Pivot operation	Turns quarter or category values into columns.
Sorter / Top k Selector	Analytical query	Ranks stores or products.
Chart / Table View	Reporting	Displays management results.
Excel Writer / CSV Writer	Report output	Exports a reusable management report.

9.3 Limitation at one million rows

KNIME workflows that materialise large intermediate tables in memory can become slow or memory-intensive as rows grow beyond one million. Database-side filtering and aggregation, column pruning, streaming-compatible nodes and incremental processing reduce this pressure. For very large data, push GROUP BY and joins to the database rather than moving all raw rows into KNIME.

Evidence to attach: annotated KNIME workflow screenshot, node configuration screenshots, output report, and a short note stating the number of rows processed and the machine/database environment.

10. TASK 10 – Recommend and Defend the Final Design

The recommended FreshCart solution is an enterprise data warehouse with dependent subject-area marts, a star schema for the principal sales and returns facts, SCD Type 2 customer history, controlled ETL and a Streamlit/SQL/KNIME reporting layer.

Area	Recommendation	Reason
Architecture	Enterprise warehouse with dependent marts	One governed definition of net sales and conformed dimensions across stores and online sales.
Schema	Star schema for reporting	Fewer joins and easier business-user navigation; snowflake only if measured hierarchy storage/governance benefits justify it.
Facts	FactSales and FactReturns	Different business events and event dates are represented at their correct grains.
History	SCD Type 2 Customer	Preserves address and loyalty tier at the time of each order.
Performance	Selective indexes and small set of aggregate tables	Accelerates frequent queries while controlling index/summary maintenance cost.
Tools	MySQL + Python + Streamlit + KNIME	Accessible open-source implementation and demonstrable ETL/OLAP operations.

The recommendation must be defended with the measured Task 4 storage/join comparison and Task 8 query tests. At one million fact rows, a relational star schema with suitable indexes is a practical starting point. Beyond that scale, partitioning the fact table by date, incremental loads, summary tables and database tuning should be evaluated. No claim of measured performance should be made until the tests have been run.

11. TASK 11 – Re-evaluate and Adapt the Design

11.A Impact analysis

Area	Impact
FactSales structure	Add source-system identifier, sales channel and DeliveryPartnerKey; retain one-row-per-order-line grain.
Product dimension	Add source-system product code mapping because online product codes differ from store/POS codes.

Sree Lakshmi S V

Customer dimension	Retain Type 2 history for address and loyalty tier; add effective timestamps and source identity.
Store / delivery	Delivery partner is not the store hierarchy. Add DimDeliveryPartner and optionally delivery status/fulfilment attributes.
ETL structure	Support multiple source adapters, code crosswalks, CDC and hourly micro-batches.
Cube	Add Channel and Delivery Partner dimensions or attributes to the analytical cube; existing Time/Product/Store/Segment hierarchies remain valid.
Performance	20,000 orders/day averages about 833 orders/hour (20,000 ÷ 24). Peak-hour bursts and multiple lines per order must be tested; hourly dashboard freshness requires reliable batch completion.

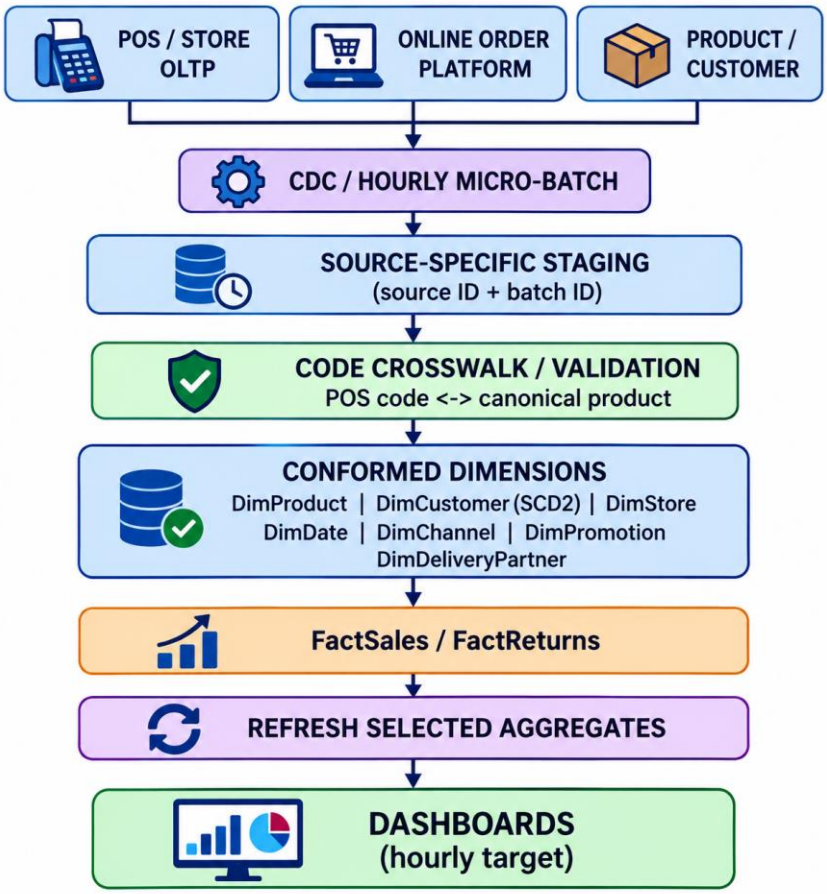
11.B Evaluate original design

The original star schema remains suitable as the analytical foundation, but its original batch assumptions and single-source product lookup are insufficient without modification. If the original ETL runs once per day, it cannot meet an hourly refresh requirement. If product codes are treated as globally unique without a source-system namespace, POS and online codes may collide. If customer address and loyalty tier are overwritten, historical sales will be attributed to the latest values rather than the values at order time.

Volume calculation: 20,000 orders per day / 24 ≈ 833 orders per hour on average. If an average order has 2.5 lines, the expected average is approximately 2,083 fact lines per hour. This is a planning estimate; actual order-line distribution and peak traffic must be measured. A 60-minute refresh target also requires time for extraction, validation, dimension processing, fact loading, aggregate refresh and dashboard query.

Sree Lakshmi S V

11.C Revised design and ETL flow



Revised dimension and key rules

- Product identity: create a ProductSourceMap table with SourceSystem, SourceProductCode, CanonicalProductBusinessID, EffectiveFrom and EffectiveTo. Map both POS and online codes to one conformed DimProduct member.
- Customer identity: use a stable enterprise CustomerBusinessID where available. If online and store identifiers differ, maintain a source-to-enterprise customer cross-reference; do not merge solely by name.
- Customer Type 2: when address or loyalty tier changes, close the current version and insert a new version. At order ingestion, resolve the CustomerKey version valid at the order timestamp.
- Delivery partner: add DimDeliveryPartner with DeliveryPartnerKey, partner code, partner name, service type and active dates. FactSales stores DeliveryPartnerKey; use an Unknown / Not Applicable member for store purchases without delivery.
- Channel: add or retain DimChannel values such as Store, Online and Mobile App. This allows channel comparisons without changing the sales fact grain.

11.D Evidence-based defence and monitoring

Run a load simulation using 20,000 orders/day and a realistic average order-line count. Generate an hourly batch and record extraction time, staging time, dimension lookup/SCD time, fact insert time,

aggregate refresh time and end-to-end freshness. Compare the result with the one-hour target. Also measure query latency before and after the selected aggregate tables are refreshed.

Metric	Evidence to record
Hourly input orders	Target: approximately 833 average orders/hour; record actual peak and average.
Fact line rows per batch	Record actual generated/received lines; do not assume every order has one line.
End-to-end load time	____ minutes; target must be less than the hourly refresh window with safety margin.
Rejected / duplicate rows	____ rows and percentage; explain top rejection reasons.
SCD2 versions created	____ customer versions per batch; verify no overlapping effective periods.
Dashboard freshness	Latest source event timestamp vs dashboard timestamp: ____ minutes.
Query latency	Record Q1/Q2/Q3 before and after aggregate refresh: ____ ms.

Monitor batch completion time, source-to-warehouse lag, late-arriving records, reject rate, duplicate rate, SCD2 version growth, aggregate refresh status, fact-table growth, index size, query latency and database CPU/memory. Alert if a batch misses the hourly deadline or if source and warehouse reconciliation differs.

12. Conclusion

The proposed FreshCart data warehouse separates analytical workloads from operational checkout processing and provides consistent, historical and multidimensional sales information. A central enterprise warehouse with dependent data marts, a star schema, conformed dimensions, separate sales and returns facts, and SCD Type 2 customer history supports the required management questions. The OLAP cube provides roll-up, drill-down, slice, dice and pivot operations. Python and Streamlit demonstrate the analytical application, while KNIME demonstrates an open-source ETL and reporting workflow. The Day-4 changes can be handled by extending the existing design with source mapping, delivery partner and channel dimensions, and hourly incremental loading. Final claims about performance must be supported by actual measurements from the implementation.

13. References

- Kimball, R. and Ross, M., The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd Edition, Wiley.
- Inmon, W. H., Building the Data Warehouse, 4th Edition, Wiley.
- KNIME Analytics Platform documentation: https://docs.knime.com/
- Streamlit documentation: https://docs.streamlit.io/
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- Python pandas documentation: https://pandas.pydata.org/docs/
- Plotly Python documentation: https://plotly.com/python/

Submission checklist

- Replace all cover-page placeholders.

Sree Lakshmi S V

- Confirm date interpretation and currency conversion policy with the assignment evaluator/source data owner.
- Run the SQL DDL on your database and include successful execution screenshots.
- Generate the 1,000 / 10,000 / 100,000 / 1,000,000 row datasets.
- Run all five OLAP operations from interactive controls and capture screenshots.
- Fill Task 8 performance tables with real timings, EXPLAIN plans, rows read and storage/load overhead.
- Build the KNIME workflow and insert annotated screenshots plus exported report.
- Run the 20,000-order/day hourly-load simulation and fill Task 11 evidence.
- Check page breaks, table readability, spelling and reference formatting before submission.

Sree Laksha S V